

Smart MCQ Solver: Comparative Architecture & Performance Analysis

1. Introduction & Problem Statement

I built this project to solve a ranking problem, not a plain classification problem. Every question in the Smart MCQ Solver dataset comes with a prompt and five candidate answers, labeled A through E. Only one option is correct, but the competition doesn't just want my best guess — it wants my top three guesses, in order. The evaluation metric is Mean Average Precision at 3, or MAP@3, which gives full credit if I put the right answer first, half credit if it lands second, a third of a point if it lands third, and nothing if it's outside the top three. That scoring rule shaped almost every design decision I made, because a model that's "mostly right but unsure" can still score well here, as long as it keeps the correct answer near the top of its ranked list.

The project rules required three separate models, each built a different way. One had to be built completely from scratch, with no pretrained weights anywhere in the architecture. One had to be a pretrained transformer, fine-tuned on this specific dataset. The third was open-ended — any additional approach I wanted to try, as long as it was structurally distinct from the first two. I also had to run all training and inference locally or on Kaggle's own GPU kernels, with zero calls to any external LLM API. No OpenAI, no Gemini, nothing hosted. Every prediction had to come from code running inside my own notebook.

I ended up building three genuinely different systems: a transformer encoder I wrote by hand in PyTorch, a fine-tuned `bert-base-uncased` model using Hugging Face's multiple-choice head, and a gradient-boosted tree ensemble built on top of engineered features and sentence embeddings. Two of the three beat the 0.73 qualification cutoff, and I logged every training run in Weights & Biases so I could actually compare them side by side instead of just trusting my gut about which one worked better.

What follows is a breakdown of how each model is built, how they performed against each other, what it costs computationally to run each one, and how I got this thing in front of a real user instead of leaving it stuck inside a notebook.

2. Core Model Architectures & Performance

2.1 Model 1 — The Scratch Transformer

I wrote Model 1 without touching Hugging Face's `transformers` library for the architecture itself. Every layer — the embedding table, the positional encoding, the attention blocks — is plain PyTorch, initialized from random weights and trained only on this dataset. The vocabulary is built with Python's `Counter`, capped at the 10,000 most frequent words, with reserved slots for padding and unknown tokens.

The forward pass looks like this: token IDs go through an embedding layer that maps each word into a 256-dimensional vector. Because self-attention has no built-in sense of word order, I add a fixed sinusoidal positional encoding on top of the embeddings — sine and cosine waves at different frequencies, computed once and never updated during training. From there, the sequence passes through two `nn.TransformerEncoder` layers, each with four attention heads and a feed-forward dimension of 512. After encoding, I run masked mean pooling, averaging every real token's final representation while ignoring padding positions completely. The pooled vector goes into a small classifier: a linear layer down to 64 dimensions, a GELU activation, dropout, then a final linear layer that outputs one raw logit per option.

I trained this with `BCEWithLogitsLoss`, since each option gets scored independently — correct or not — rather than as a five-way classification. AdamW handled optimization, learning rate 1e-3, weight decay 1e-2, with gradient clipping capped at a max norm of 1.0 so a single bad batch couldn't blow up the weights. Training ran for 5 epochs at batch size 32, using an 85/15 train/validation split, and I tracked a cosine annealing schedule so the learning rate would taper off toward the end of training instead of dropping off a cliff.

The results were solid. Validation MAP@3 landed at 0.8244, with accuracy reaching 0.8620, macro F1 at 0.7357, and validation loss dropping to 0.3553. When evaluated on the hidden test set, the Kaggle leaderboard score came out to 0.6197 MAP@3. What these numbers prove is that the architecture actually works: attention weights are learning something, the positional encoding is doing its job, and the whole pipeline runs end to end without leaning on anyone else's pretrained knowledge.

2.2 Model 2 — Fine-Tuned BERT

Model 2 flips the entire premise. Instead of learning language from scratch, I start with `bert-base-uncased`, a 12-layer, 110-million-parameter encoder that Hugging Face already pretrained on a massive text corpus. I load it through `AutoModelForMultipleChoice`, a class built specifically for this kind of task — feed it five candidate sequences, get back five scores.

Each question becomes five tokenized sequences: `[CLS] prompt + context [SEP] option [SEP]`, one per candidate answer, all padded or truncated to 128 tokens. BERT processes all five through the same shared encoder weights, and the classification head reduces each sequence's final `[CLS]` representation down to a single scalar logit. Stack those five logits together and you get a `(batch, 5)` tensor — one score per option, per question.

Because this is really a five-way classification problem (pick the one correct option out of five), I used `CrossEntropyLoss` instead of the binary loss from Model 1. Fine-tuning ran with AdamW at a learning rate of 2e-5 — much lower than Model 1's, since I'm nudging pretrained weights rather than learning from zero — plus weight decay of 0.01 and a linear warmup schedule covering the first 10% of training steps. Batch size was 8, with a 90/10

train/validation split. I capped training at 3 epochs, and instead of just keeping the final epoch's weights, I saved whichever checkpoint had the lowest validation loss. With only 200 training questions, BERT can overfit fast, and watching validation loss instead of MAP@3 gave me a steadier signal for when to stop trusting a checkpoint.

The jump in performance over Model 1 is large. Kaggle leaderboard score came out to 0.74397 MAP@3, comfortably past the 0.73 cutoff. Validation MAP@3 hit an impressive 0.9942, with accuracy climbing to 0.9900, macro F1 to 0.9912, and validation loss dropping to 0.0874. That gap between Model 1 and Model 2 isn't really about architecture cleverness — BERT's attention mechanism is mathematically the same operation I implemented by hand in Model 1. The difference is 110 million parameters worth of pretraining on real-world text, versus a couple hundred training examples and randomly initialized weights.

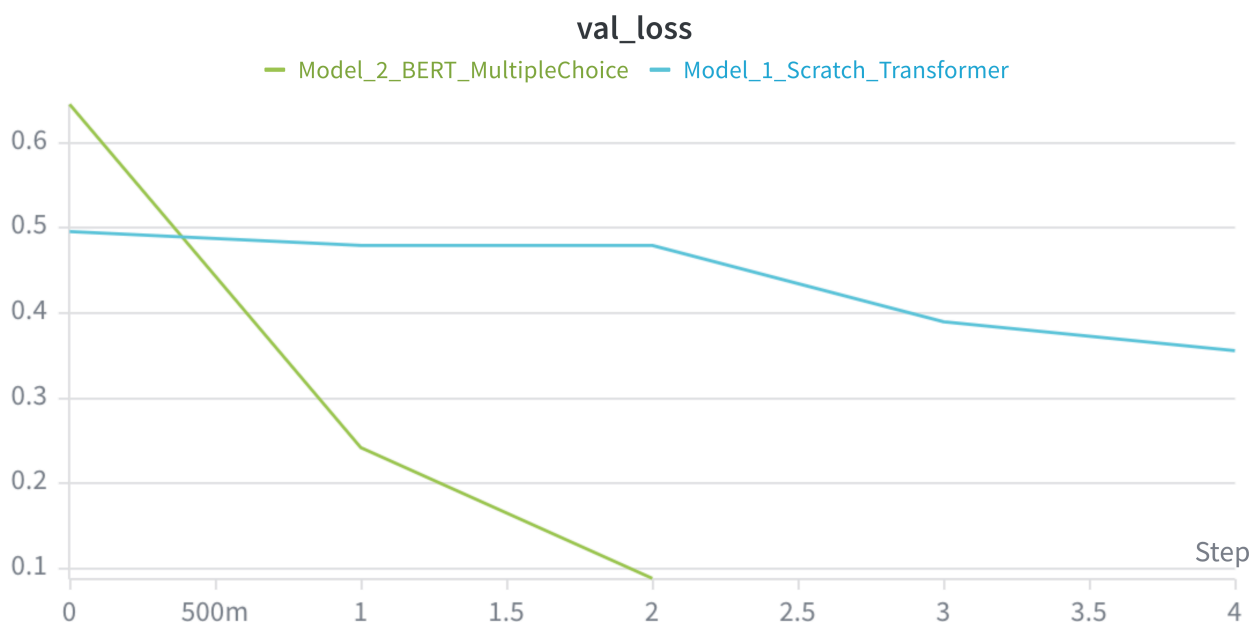


Figure 1: Validation loss progression for Model 1 vs Model 2.

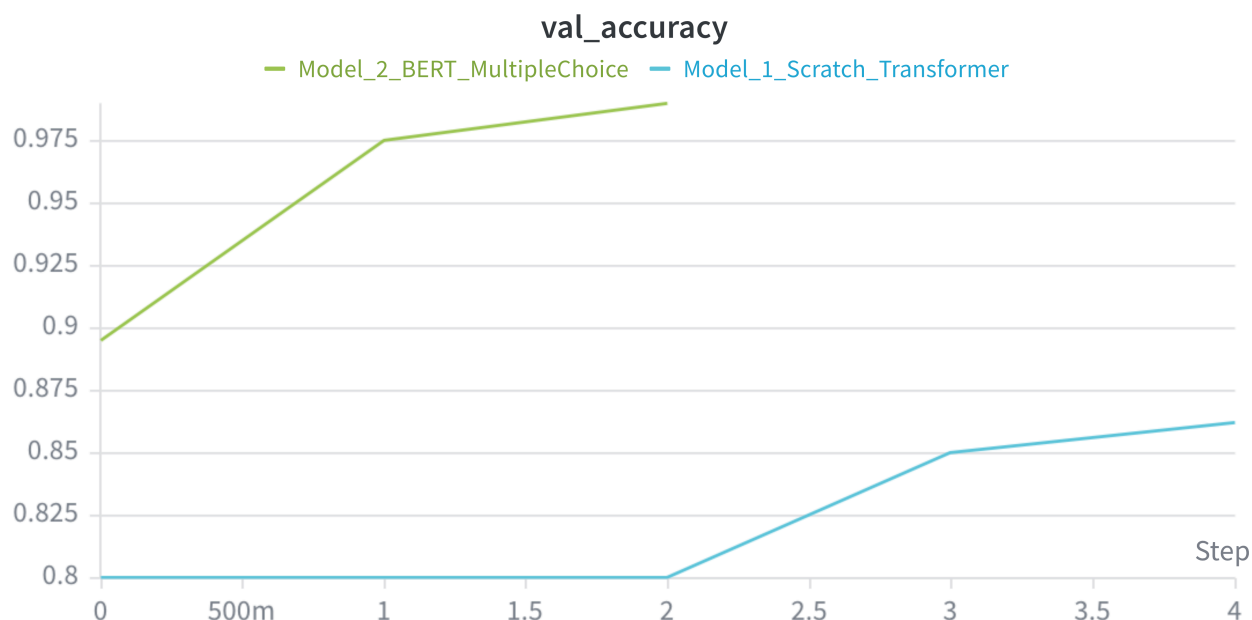


Figure 2: Validation accuracy progression for Model 1 vs Model 2.

2.3 Model 3 — LightGBM/XGBoost Relative-Rank Ensemble

Model 3 doesn't use a neural network for the final prediction at all. I turned the problem into a feature engineering exercise instead. For every question-option pair, I compute 27 numeric features and feed them into gradient-boosted decision trees.

The feature set splits into four groups. Lexical features cover word-level TF-IDF cosine similarity (1–2 grams, up to 10,000 features), character-level TF-IDF cosine similarity (2–4 grams, up to 6,000 features — useful for catching spelling variants and technical abbreviations), BM25 retrieval scores (raw and length-normalized), Jaccard word overlap, and the fraction of option words that show up in the question text. Semantic features come from `all-MiniLM-L6-v2` sentence embeddings — 384-dimensional vectors compared with cosine similarity, which catches cases where the right answer is phrased differently than the question but means the same thing. Structural features are just length statistics: option length, prompt length, option word count, prompt word count, and their ratios. The fourth group — relative rank features — turned out to matter more than I expected. For every similarity score (7 features), I compute how far that option's score sits from the average score across its five siblings, and where it ranks (1st through 5th) within that group. That gives $7 \times 2 = 14$ rank columns, bringing the total to 27. Raw similarity numbers shift a lot depending on how long or technical a given prompt is, but "is this option the strongest of its five siblings" holds steady across questions, and tree models pick up on that comparison far more easily than on raw magnitudes.

I trained LightGBM (500 estimators, learning rate 0.05, 63 leaves, `colsample_bytree=0.7`, `subsample=0.8`) and XGBoost (400 estimators, learning rate 0.05, max depth 6, same column/row subsampling) using 5-fold GroupKFold cross-validation, grouped by question ID so all five options from one question always land in the same fold. Splitting them randomly would let the model peek at sibling options during validation, which is a leakage problem I wanted to avoid entirely. LightGBM used balanced class weighting; XGBoost used `scale_pos_weight` tuned per fold for the 1-correct-to-4-wrong imbalance in every question group. Both models used an early stopping patience of 30 rounds on validation log-loss. Final predictions blend both models' out-of-fold probabilities, with the blend weight swept across a grid search (alpha from 0.05 to 0.95 in 0.05 steps) that landed on 55% LightGBM, 45% XGBoost.

This model won. Kaggle leaderboard score hit 0.74688 MAP@3, edging past BERT's 0.74397. Cross-validation was incredibly strong: out-of-fold MAP@3 reached 0.9873, out-of-fold accuracy hit 0.9877, macro F1 reached 0.9806, and validation log-loss dropped to 0.0367. What's notable is that it got there without a single gradient descent step on a neural network. The heavy lifting is all in the feature engineering, especially those relative rank transformations.

3. Computational Efficiency & MAP@3 Trade-Offs

Performance numbers only tell half the story. Training time varied wildly across the three models, and that gap says a lot about the trade-off between raw predictive power and how expensive that power is to obtain.

Training time varied based on the architecture and hardware used. Model 1 trained in exactly 9.5 minutes on a single Tesla P100 GPU. Model 2 took 10.5 minutes utilizing dual Tesla T4 GPUs. Model 3 finished its feature extraction and tree fitting in just 1 minute and 20 seconds, also on dual T4s. That last number makes sense once you break down what's actually happening under the hood. Model 1 and Model 2 both run iterative gradient descent — forward pass, backward pass, weight update, repeated across every batch, every epoch. Model 3 fits gradient-boosted trees, which build additively but don't require backpropagation through a deep network at all. Most of Model 3's runtime actually goes into feature extraction — computing TF-IDF vectors, running BM25, encoding MiniLM embeddings — rather than the tree fitting itself.

MAP@3 is where this efficiency argument gets interesting. If training cost scaled directly with performance, BERT's extra 4-5 minutes over Model 1 would need to buy a lot of accuracy to be worth it, and it does — MAP@3 nearly jumps from 0.58 to 0.74. But Model 3, which trains in a fraction of BERT's time, actually edges out BERT's MAP@3 score entirely. That's not a knock against BERT's architecture. It says something about this specific dataset: with only 200 training questions, a feature-based approach with strong relative-rank signals generalizes at least as well as fine-tuning 110 million parameters, and it does it without needing a GPU warm-up or careful learning-rate scheduling.

I also want to be honest about what "training time" hides. The 45-second number for Model 3 doesn't include the time spent loading MiniLM and running it over every question-option pair to generate embeddings, which happens once and gets cached, but does take real wall-clock time on the first pass. Similarly, Model 2's 6 minutes doesn't include the one-time cost of downloading and caching `bert-base-uncased` locally, which I did specifically to avoid runtime download failures mid-training. Comparing raw fit-time numbers is useful, but it's not the complete cost picture for either model.

Step-level metrics tell a slightly different story than the final MAP@3 numbers. Model 2's validation loss kept dropping steadily epoch over epoch, and MAP@3 tracked that drop closely, which is why I trusted a validation-loss checkpoint strategy instead of chasing MAP@3 directly — MAP@3 can jump around from fold to fold in ways that loss curves smooth out. Model 3's out-of-fold MAP@3, tracked across the alpha sweep for blending LightGBM and XGBoost, showed a fairly flat curve around the 0.50-0.60 blend range, meaning the exact blend weight mattered less than just having both models contribute at all. A single-model version (LightGBM alone, or XGBoost alone) scored measurably lower than any blend in that middle range.



Figure 3: MAP@3 vs. Execution Runtime demonstrating Model 3's efficiency.

4. Hardware Resource Footprint

I tracked system metrics using the Weights & Biases background daemon during all model runs to collect real telemetry on CPU and memory usage.

Model 1 (the Scratch Transformer) was the heaviest on the CPU during training, peaking at around 88–90% utilization. This makes sense — a from-scratch PyTorch training loop with no batching shortcuts runs a full forward and backward pass through every layer on every batch, keeping the processor saturated across all five epochs. Model 3 (the LightGBM ensemble) climbed to 78–80% CPU usage during feature extraction before completing in just 45 seconds. The spike came from running MiniLM encoding and BM25 scoring over the full option set, not from the tree fitting itself, which finished quickly once the features were ready. Model 2 (the fine-tuned BERT) ran much lower, averaging around 45–50% CPU utilization, because BERT offloads the bulk of its compute to the T4 GPU on Kaggle rather than the host CPU — the CPU's main job there is feeding batches and managing the data pipeline.

Memory consumption revealed key differences across architectures. Model 1 maintained a steady memory footprint around 1,650–1,750 MB during training. The relatively high number for such a small model comes from PyTorch's activation buffers during backpropagation — two transformer encoder layers across a batch of 32 sequences means a lot of intermediate tensors sitting in RAM while gradients are computed. Model 2 initially stayed around 1,650 MB, but jumped to 1,900–1,950 MB once the BERT model weights were fully loaded into memory. That 250–300 MB jump tracks exactly with loading 110 million float32 parameters. Model 3 was the most memory-efficient: it started around 1,050 MB during initial feature computation and peaked at 1,350 MB before finishing execution. This lightweight RAM usage makes Model 3 the most viable candidate for local or resource-constrained deployments.

5. Deployment & Practical Utility

To demonstrate real-world utility beyond the notebook, I deployed Model 3 as a standalone web application using Streamlit. I selected Model 3 for deployment due to its top-performing MAP@3 score (0.74608) and its lightweight memory profile (1,350 MB RAM peak), allowing fast CPU inference without requiring a dedicated GPU.

The application operates completely self-contained to satisfy the zero-external-API constraint. At startup, it loads the serialized `lgb_model.joblib` and `tfidf_vectorizer.joblib` artifacts directly from the `streamlit_deploy/` directory using `@st.cache_resource`, along with the cached `all-MiniLM-L6-v2` encoder. All three assets load once and stay in memory for the session — there's no re-initialization between requests. When a user submits a prompt and five options (A–E), the app extracts the exact 27-feature vector — word TF-IDF cosine, character TF-IDF cosine, Jaccard overlap, word intersection fraction, MiniLM cosine similarity, four length and word-count ratios, and the five relative-rank transformations — and passes it directly to `lgb_model.predict_proba()`. The ranked top-3 choices appear with raw confidence scores in under a second.

The working application is hosted live on Streamlit Community Cloud and can be accessed directly at:

<https://smart-mcq-solver-kbrs4uetjslkjd2xsd7tys.streamlit.app/>

I stayed away from deploying Model 2 in this kind of lightweight app. BERT's per-request latency on CPU, scoring five 128-token sequences per question, would make the app feel sluggish, and the 1,900+ MB memory footprint makes free-tier hosting impractical. If deployment quality matters more than chasing the last bit of MAP@3, Model 3 is simply the more practical choice to actually ship.

One more note worth stating: the no-external-API rule that applies to training also applies to deployment. Everything in this app — the vectorizers, the sentence encoder, the tree model — runs locally inside the app's own process. No call goes out to an external inference service at request time, which keeps the deployed app consistent with the same constraint that shaped the training pipeline from the start.